App móvil Ferretería Pescado Volador

Documentación de la app: Laboratorio 11, puntos 1 a 9

Proyecto	`Pescado_Volador_Movil` (variante Vue de Ionic)
Tecnologías	Ionic 9 · Vue 3.5 · Vue Router 5 · Capacitor 8 · Pinia 4 · axios 1.21 · TypeScript 5.9
API que consume	`FerreteriaPescadoVolador` (Laravel + Sanctum), token de 2 horas
Entidades	Productos (CRUD completo), Ventas (mismas funciones que el front web) y Recepción de compras (cámara)
Hardware	Cámara, con `@capacitor-mlkit/barcode-scanning` (códigos de barras Code 128)
Plataforma nativa	Android, appId `com.pescadovolador.movil`
Fecha	8 de noviembre de 2026

Contenido

1. Resumen: qué se hizo y dónde está cada requerimiento
2. Arquitectura de la app (capas y carpetas)
3. Flujo de datos de la app móvil
4. Punto 1: pantallas e identidad visual
5. Punto 2: enrutamiento, guardián y botón atrás
6. Punto 3: punto único de intercepción (token y 401)
7. Punto 4: consumo de la API
8. Punto 5: token en almacenamiento cifrado
9. Punto 6: caché sin conexión y detección de red
10. Punto 7: cámara, recepción de compras y permisos
11. Punto 8: estados de carga, vacío y error
12. Punto 9: tamaños de pantalla y áreas seguras
13. Decisiones de diseño móvil, complementos y manejo de permisos
14. Permisos por rol
15. Configuración y cómo ejecutar
16. Pruebas realizadas
17. Limitaciones conocidas

Los nombres de archivo son relativos a `Pescado_Volador_Movil/`. Los fragmentos de código son extractos simplificados del código real.

1. Resumen: qué se hizo y dónde está

La tabla resume cada requerimiento del enunciado, su estado y los archivos donde está la implementación principal. Las secciones 4 a 12 explican cada uno con detalle.

Punto	Requerimiento (resumido)	Archivos principales	Estado
1 Pantallas	Inicio de sesión, listado, detalle y formulario con componentes de Ionic. Identidad visual del front.	`views/LoginPage.vue`, `views/productos/*`, `views/ventas/*`, `theme/variables.css`, `theme/marca.css`	Hecho
2 Rutas	Carga diferida, parámetros de ruta, guardián y botón atrás que no vuelve al login.	`router/index.ts`, `router/guardian.ts`, `App.vue`	Hecho*
3 Interceptor	Un solo punto adjunta el token y, ante un 401, cierra la sesión y va al login.	`servicios/api.ts`, `servicios/errores.ts`, `stores/sesion.ts`, `App.vue`	Hecho
4 API	Login, listado con scroll infinito y recarga al deslizar, detalle y escritura.	`servicios/authApi.ts`, `servicios/productosApi.ts`, `servicios/ventasApi.ts`, `stores/productos.ts`, `stores/ventas.ts`	Hecho
5 Token cifrado	Token en almacenamiento cifrado del dispositivo, nunca en Preferences ni en localStorage.	`servicios/almacenSeguro.ts`, `stores/sesion.ts`	Hecho
6 Sin conexión	Caché del último listado, plugin de red y aviso de datos no actualizados con su marca de tiempo.	`servicios/cache.ts`, `stores/red.ts`, `components/AvisoDatosGuardados.vue`, `components/AvisoSinConexion.vue`	Hecho*
7 Hardware	Cámara con un plugin de Capacitor; permiso declarado, pedido en tiempo de ejecución y caso de denegación permanente.	`servicios/escaner.ts`, `composables/useEscanerCodigo.ts`, `views/recepcion/RecepcionPage.vue`, `stores/recepcion.ts`, `AndroidManifest.xml`	Hecho*
8 Estados	Carga, vacío y error con reintento en todas las páginas que consumen datos, con componentes de Ionic.	`components/EstadoCarga.vue`, `EstadoError.vue`, `EstadoVacio.vue`, `MensajeAviso.vue` y todas las páginas	Hecho
9 Pantallas	Dos tamaños de pantalla y áreas seguras del dispositivo.	Componentes de Ionic + `viewport-fit=cover` (`index.html`) + manejo de insets de Capacitor 8; `theme/escaner.css`	Hecho*

* Implementado y verificado en el navegador contra la API real y con pruebas unitarias. La comprobación en un dispositivo Android (botón atrás, cifrado real, modo avión, diálogos de permiso, cámara y áreas seguras) la realiza el equipo en un teléfono físico.

Decisiones tomadas con el equipo

Tema	Decisión
Navegación	Pestañas inferiores: **Productos**, **Ventas** y **Recepción** (cada una visible según el permiso).
Estado de la app	Pinia (almacenes o *stores*).

Tema	Decisión
Botón atrás en la pantalla raíz	Pide confirmación: *«¿Desea salir de la aplicación?»*.
Tema	Claro, como el front, con un botón para pasar a modo oscuro; la elección se recuerda.
Cliente HTTP	axios con interceptores.
Almacenamiento cifrado	`@aparajita/capacitor-secure-storage`.
Listados	Desplazamiento infinito **y** recarga al deslizar hacia abajo.
Caché	Productos y Ventas; se borra al cerrar sesión o al expirar el token. Productos guarda el listado sin búsqueda; sin conexión, la búsqueda filtra en el dispositivo.
Sin conexión	Se bloquean las escrituras; el detalle se lee de la caché.
Permisos	Se ocultan las acciones que el rol no puede hacer (según las *abilities* del token); un 403 se muestra con su mensaje.
Expiración (401)	Se conserva el formulario de venta y se recupera después de volver a iniciar sesión, igual que el front.
HTTP en Android	Contenido mixto permitido y HTTP sin cifrar limitado a las IPs de desarrollo.
Hardware (punto 7)	**Cámara** para la **recepción de compras** que implementa el backend: escanear, elegir la compra pendiente y confirmar. El stock lo suma el backend y queda en el Kardex.
Formato del código	Solo **Code 128**, el formato que genera el backend en `/products/{id}/barcode`.
Sin cámara	Siempre se puede escribir el código. En el navegador solo existe esa opción. Con el permiso bloqueado se ofrece **Abrir ajustes**.
Cantidad distinta	Se avisa y se desactiva «Confirmar recepción» hasta que coincida con la esperada.
Confirmación	Diálogo antes de recibir, porque no se puede deshacer.
Estados (punto 8)	Todos los mensajes con componentes de Ionic, en forma de **tarjeta** (`ion-card`) con ícono y botón.

2. Arquitectura de la app

La app está organizada en **capas**. Cada capa solo habla con la de abajo, así cada archivo tiene una sola responsabilidad y es fácil encontrar dónde se hace cada cosa.

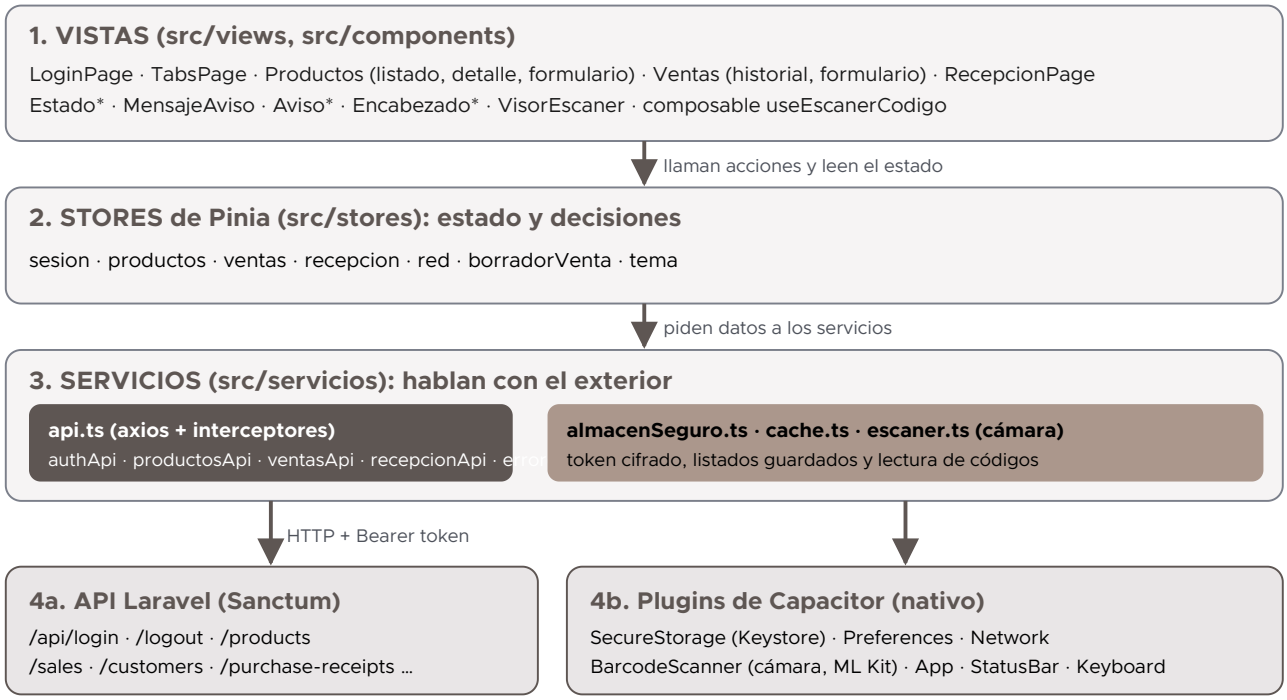


Figura 1. Capas de la app. Solo `api.ts` hace solicitudes HTTP. El token y la caché se guardan a través de `almacenSeguro.ts` y `cache.ts`, y la cámara solo se usa desde `escaner.ts`. `stores/red.ts` (Network) y `stores/tema.ts` (Preferences) usan su plugin directamente.

¿Qué hace cada carpeta?

Archivo	Responsabilidad
`src/main.ts`	Arranque: instala Ionic, Pinia y el router; carga la fuente, el tema guardado y el estado de la red antes de mostrar la app.
`src/App.vue`	Raíz. Maneja el botón atrás de Android y lleva al login cuando la sesión expira.
`src/router/index.ts`	Rutas con carga diferida, parámetros `:id` y permisos (`meta.permiso`).
`src/router/guardian.ts`	`resolverAcceso()`: decide si una navegación se permite o a dónde redirige. `destinoTrasLogin()` valida el `?redirect=`.
`src/servicios/api.ts`	**Punto único** de salida a la API: instancia de axios con los interceptores de request y response.
`src/servicios/errores.ts`	`ErrorApi` y `aErrorApi()`: convierte cualquier error en uno uniforme (status, mensaje y errores por campo).
`src/servicios/authApi.ts`	`iniciarSesionApi()` (POST /login) y `cerrarSesionApi()` (POST /logout).
`src/servicios/productosApi.ts`	CRUD de productos, catálogos y adaptadores (inglés ↔ español).
`src/servicios/ventasApi.ts`	Historial, crear y editar ventas; clientes; adaptador de montos.

Archivo	Responsabilidad
`src/servicios/almacenSeguro.ts`	Guarda, lee y borra la sesión (token) en el almacenamiento **cifrado**.
`src/servicios/cache.ts`	Guarda y lee los listados con su marca de tiempo (Preferences).
`src/stores/sesion.ts`	Token en memoria, permisos, restaurar al abrir, abrir, cerrar y expirar la sesión.
`src/stores/productos.ts` · `ventas.ts`	Listados paginados, uso de la caché sin conexión, detalle y escrituras.
`src/stores/red.ts`	Estado de la conexión con el plugin `@capacitor/network`.
`src/stores/borradorVenta.ts`	Conserva el formulario de venta si la sesión expira a mitad de la captura.
`src/stores/tema.ts`	Tema claro u oscuro, recordado con Preferences.
`src/servicios/recepcionApi.ts`	Punto 7: producto por código, compras pendientes y confirmar la recepción.
`src/servicios/escaner.ts`	Punto 7: **único** archivo que usa el plugin de la cámara (permiso, escaneo y Ajustes).
`src/composables/useEscanerCodigo.ts`	Punto 7: flujo del permiso en pantalla (explicar, pedir, bloqueado), botón atrás y app en segundo plano.
`src/stores/recepcion.ts`	Punto 7: estado de la recepción (producto, pendientes, elegido, errores) y confirmación.
`src/components/Estado*.vue`, `MensajeAviso.vue`	Punto 8: estados de carga, vacío y error, y mensajes, con componentes de Ionic.
`src/types/modelos.ts`	Tipos: Producto, Venta, Cliente, DetalleRecepcion, ElementoCatalogo, Pagina, ErroresCampo.
`src/utils/*`	`formato.ts` (colones, fechas), `conversion.ts` (texto → número), `navegacion.ts` (volver y avisos tipo *toast*).
`src/theme/*`	Paleta del front mapeada a Ionic (`variables.css`), estilos de marca (`marca.css`) y la capa del escáner (`escaner.css`).

3. Flujo de datos de la app móvil

La figura 2 sigue un dato desde que se abre la app hasta que se ve en pantalla, usando como ejemplo el listado de productos. Los números marcan el orden.

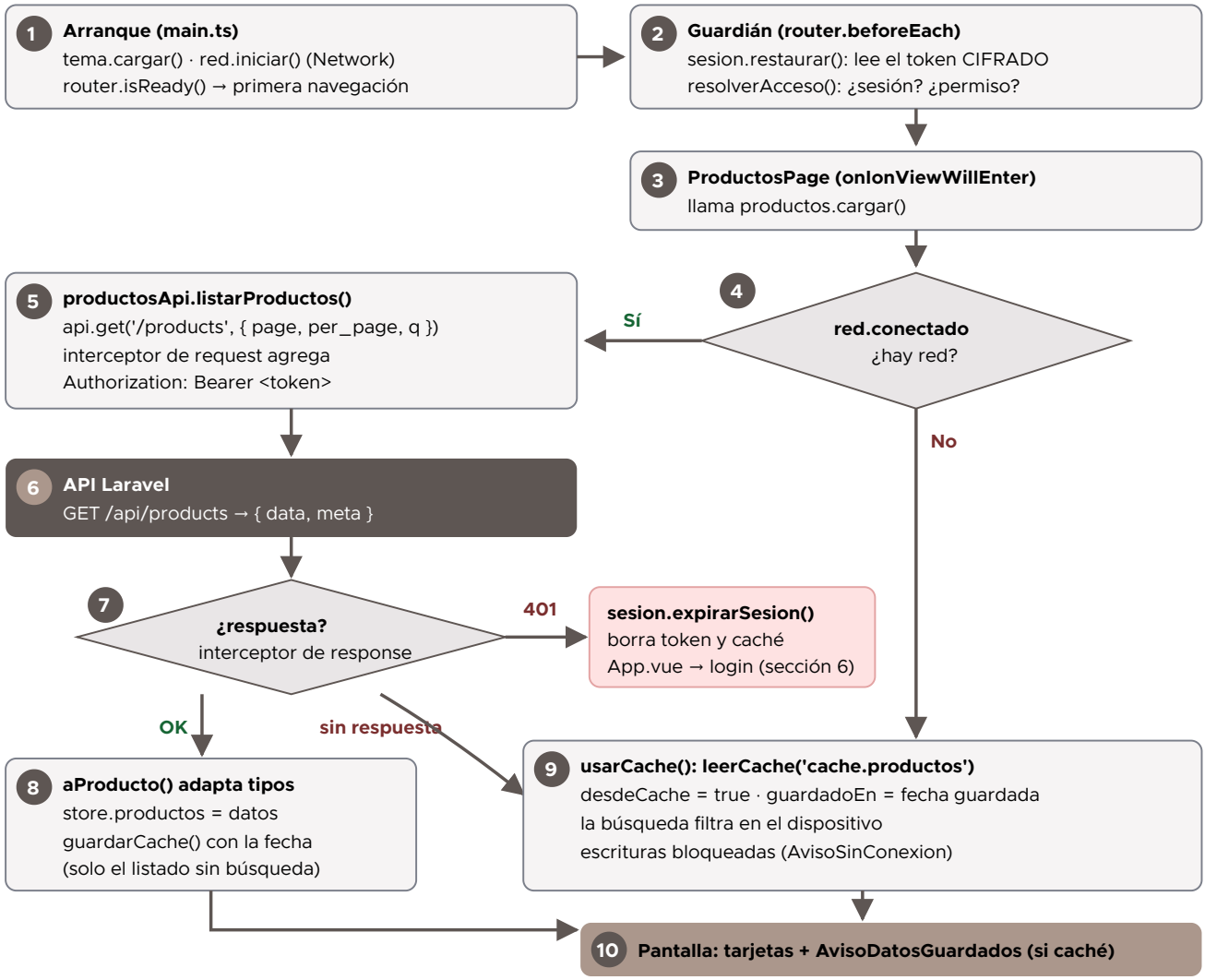


Figura 2. Flujo de datos al abrir el listado de productos (Ventas funciona igual con `stores/ventas.ts`).

Lo mismo, contado paso a paso

1. **Arranque** (`main.ts`): se aplica el tema guardado, se consulta el estado de la red con `Network.getStatus()` y se espera la primera navegación.
2. **Guardián**: antes de entrar a cualquier página, `sesion.restaurar()` lee el token del almacenamiento cifrado (solo la primera vez) y `resolverAcceso()` decide si se entra o se redirige.
3. **Página**: al mostrarse, la página le pide los datos al *store*. Las páginas nunca llaman a la API directamente.
4. **¿Hay red?** El *store* consulta `red.conectado`.
5. Con red, el **servicio** arma la solicitud y la manda por `api`. El **interceptor de request** agrega el token.
6. La **API** responde con `{ data, meta }` (paginación de Laravel).

7. El **interceptor de response** revisa la respuesta: si es 401 cierra la sesión; cualquier error se convierte en `ErrorApi`.
8. Si todo salió bien, el **adaptador** corrige los tipos, el *store* guarda los datos y los copia a la **caché** con la hora.
9. Sin red (o si el servidor no responde), el *store* lee la **caché** y marca que los datos no están actualizados.
10. La **pantalla** se actualiza sola (reactividad de Vue). Si los datos vienen de la caché, muestra el aviso con la fecha.

Flujo de una escritura (crear, editar o eliminar)

`ProductoFormularioPage` → `productos.crear(cuerpo)` → `crearProducto()` traduce los campos al inglés (`aCuerpoApi`) → `api.post('/products')` con el token → si la API responde **422**, `traducirErrores()` pasa las claves al español y el formulario marca cada campo; si responde **201**, se muestra un aviso y se vuelve atrás, y el listado se recarga al volver a mostrarse (`necesitaRecarga`). Sin conexión, el botón de guardar está desactivado.

4. Punto 1: pantallas e identidad visual

Las pantallas

Pantalla	Archivo	Componentes de Ionic y función
Inicio de sesión	`views/LoginPage.vue`	`ion-input` con `ion-input-password-toggle`, `ion-button` y `ion-spinner`. Errores por campo (422) y mensaje general (401, 429 o sin conexión).
Contenedor de pestañas	`views/TabsPage.vue`	`ion-tabs` e `ion-tab-bar`: Productos, Ventas (con `view sales`) y Recepción (con `receive purchases`).
Listado de productos	`views/productos/ProductosPage.vue`	`ion-searchbar`, `ion-card`, `ion-refresher`, `ion-infinite-scroll` e `ion-fab` para crear.
Detalle de producto	`views/productos/ProductoDetallePage.vue`	`ion-list`, `ion-item` e `ion-note`; Editar y Eliminar (con confirmación en `ion-alert`).
Formulario de producto	`views/productos/ProductoFormularioPage.vue`	Crea y edita en la misma página: `ion-input`, `ion-textarea`, `ion-select` e `ion-toggle`.
Historial de ventas	`views/ventas/VentasPage.vue`	Tarjetas con cliente, total, *badge* de estado, fecha y Editar; también recarga al deslizar y scroll infinito.
Formulario de venta	`views/ventas/VentaFormularioPage.vue`	Igual que el front: búsqueda por SKU que calcula el total, Limpiar y Cancelar edición.
Recepción de compras (punto 7)	`views/recepcion/RecepcionPage.vue`	Botón para escanear (cámara), `ion-input` para el código, `ion-list` con el producto y el stock, `ion-select` con la compra pendiente y la cantidad recibida. Sigue el diseño del formulario de recepción acordado por el equipo, con la identidad de la app.

Identidad visual (alineada con Ferreteria_Front)

- `theme/variables.css`: los colores del front (`--pizarra`, `--niebla`, `--acero`, `--arena`…) con los mismos nombres, mapeados a los colores de Ionic (`--ion-color-primary` = pizarra). Incluye una paleta oscura derivada.
- `theme/marca.css`: equivalentes de las clases del front: tarjeta (`.card`), *badges*, grillas, grupo de campos (`fieldset`), franja bajo el encabezado (roja en móvil y verde desde 48rem, igual que el front) y avisos.
- Fuente **Clarity City** incluida en la app (`@fontsource/clarity-city`), para que funcione sin internet.
- Modo oscuro: el botón luna o sol de `components/EncabezadoPrincipal.vue` llama a `tema.alternar()`, que pone la clase `.ion-palette-dark` y guarda la elección con Preferences.

5. Punto 2: enrutamiento, guardián y botón atrás

Rutas (router/index.ts)

Ruta	Página (carga diferida)	meta
`/login`	LoginPage	`publica: true`
`/tabs/productos`	ProductosPage	`permiso: 'view products'`
`/tabs/productos/nuevo`	ProductoFormularioPage	`'create products'`
`/tabs/productos/:id(\d+)`	ProductoDetallePage (`props: true`)	`'view products'`
`/tabs/productos/:id(\d+)/editar`	ProductoFormularioPage (`props: true`)	`'update products'`
`/tabs/ventas`	VentasPage	`'view sales'`
`/tabs/ventas/nueva`	VentaFormularioPage	`'create sales'`
`/tabs/ventas/:id(\d+)/editar`	VentaFormularioPage (`props: true`)	`'update sales'`
`/tabs/recepcion`	RecepcionPage	`'receive purchases'`

- **Carga diferida:** `component: () => import('@/views/…')`. Vite genera un archivo JS por página, y cada uno se descarga solo cuando se visita.
- **Parámetro de ruta:** `:id(\d+)` solo acepta números, así que `nuevo` o `nueva` nunca se confunden con un id. Con `props: true`, la página recibe `id` como *prop* (`defineProps<{ id: string }>()`).

Guardián (router/guardian.ts → resolverAcceso)

```
router.beforeEach(async (to) => {
  const sesion = useSesionStore();
  await sesion.restaurar();          // lee el token cifrado (solo la 1.ª vez)
  return resolverAcceso(to, sesion);
});

// resolverAcceso(destino, sesion):
// 1. ruta privada y sin sesión       → { path: '/login', query: { redirect } }
// 2. con sesión y destino = /login   → '/tabs/productos'  (el atrás nunca vuelve al login)
// 3. falta meta.permiso del token    → '/tabs/productos'
// 4. si no                           → true (se permite)
```

Todas las rutas son **privadas por defecto**: solo las marcadas con `meta.publica` se ven sin sesión. Así, una página nueva queda protegida aunque nadie se acuerde de marcarla.

Botón físico de retroceso (App.vue y LoginPage.vue)

1. **LoginPage** navega con `ionRouter.navigate(destino, 'root', 'replace')`: el destino **reemplaza** al login en el historial, así que no queda una entrada del login a la cual volver.
2. El **guardián** redirige al inicio si, con sesión activa, algo intenta ir a `/login`.
3. **App.vue** registra `useBackButton(-1, …)`. El router de Ionic (prioridad 0) retrocede una página y luego llama al siguiente manejador; si ya no hay a dónde volver (`!ionRouter.canGoBack()`), aparece *«¿Desea salir de la aplicación?»* y, al aceptar, `App.exitApp()` de `@capacitor/app`.

6. Punto 3: punto único de intercepción

Archivo clave: `src/servicios/api.ts`. Toda la app usa la misma instancia de axios (`api`). Ningún servicio ni pantalla adjunta el token a mano: el enunciado no lo acepta.

```
export const api = axios.create({ baseURL: URL_API, timeout: 15000,
                                  headers: { Accept: 'application/json' } });

// 1) REQUEST: adjunta el token a CADA solicitud
api.interceptors.request.use((config) => {
  const { token } = useSesionStore();
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// 2) RESPONSE: 401 → cierra la sesión; todo error → ErrorApi
api.interceptors.response.use((r) => r, async (error) => {
  const errorApi = aErrorApi(error);
  if (errorApi.status === 401 && !['/login', '/logout'].includes(error.config?.url ?? '')) {
    await useSesionStore().expirarSesion();
  }
  return Promise.reject(errorApi);
});
```

¿Por qué se excluyen `/login` y `/logout`? El backend responde **401** cuando las credenciales son incorrectas (*«Las credenciales proporcionadas son incorrectas.»*), y eso no significa que una sesión haya expirado. En `/logout`, un token que ya no servía no debe disparar la redirección, porque el usuario ya está saliendo.

Qué pasa ante un 401 (secuencia)

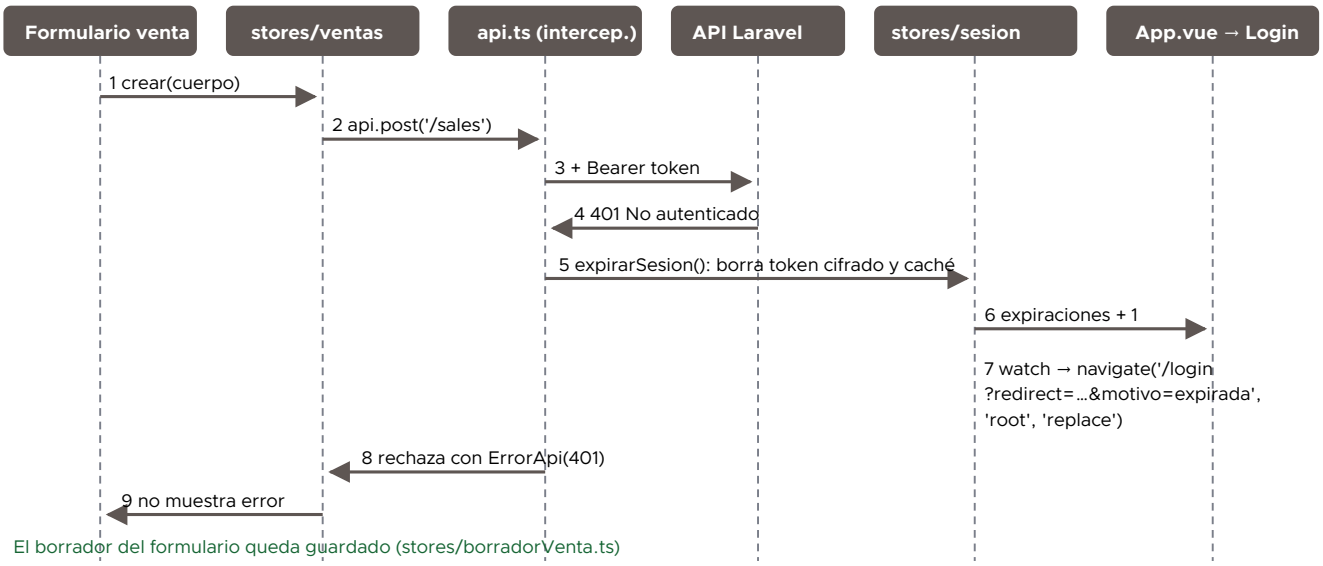


Figura 3. Expiración de la sesión. Después del login, `?redirect` devuelve al formulario con los datos recuperados.

Archivos que participan

Archivo / función	Papel
`servicios/api.ts`	Los 2 interceptores (el punto único).

Archivo / función	Papel
`servicios/errores.ts` → `ErrorApi`, `aErrorApi()`	Error uniforme: `status` (0 = sin conexión), `message` y `errores` por campo. Incluye `esDeConexion`.
`stores/sesion.ts` → `expirarSesion()`	Borra el token cifrado y la caché, y aumenta `expiraciones`. No hace nada si ya no había sesión (por eso varios 401 a la vez cuentan una sola expiración).
`App.vue` → `watch(sesion.expiraciones)`	Navega al login como nueva raíz, recordando la página actual.
`views/LoginPage.vue`	Con `?motivo=expirada` muestra: *«Su sesión expiró (el token vence a las 2 horas)…»*.
`stores/borradorVenta.ts` + `VentaFormularioPage.vue`	El formulario guarda cada cambio en el borrador. Si la página se cierra por expiración (sin sesión), el borrador se conserva y se recupera al volver: *«Sesión restaurada: no se perdieron los datos del formulario.»*

7. Punto 4: consumo de la API

Operación	Endpoint	Servicio	Store / pantalla
Inicio de sesión	`POST /login`	`authApi.iniciarSesionApi()`	`LoginPage` → `sesion.abrirSesion()`
Cerrar sesión	`POST /logout`	`authApi.cerrarSesionApi()`	`EncabezadoPrincipal` → `sesion.cerrarSesion()`
Listado productos	`GET /products?page&per_page=15&q`	`productosApi.listarProductos()`	`productos.cargar()` / `cargarMas()` → `ProductosPage`
Detalle	`GET /products/{id}`	`productosApi.obtenerProducto()`	`productos.obtener()` → `ProductoDetallePage`
Escritura: crear	`POST /products`	`productosApi.crearProducto()`	`ProductoFormularioPage`
Escritura: editar	`PUT /products/{id}`	`productosApi.actualizarProducto()`	`ProductoFormularioPage`
Escritura: eliminar	`DELETE /products/{id}`	`productosApi.eliminarProducto()`	`ProductoDetallePage`
Catálogos	`GET /categories` · `/brands` · `/units`	`listarCategorias()` …	`productos.cargarCatalogos()`
Listado ventas	`GET /sales?sort=id&direction=desc`	`ventasApi.listarVentas()`	`ventas.cargar()` / `cargarMas()` → `VentasPage`
Escritura: venta	`POST /sales` · `PUT /sales/{id}`	`crearVenta()` · `actualizarVenta()`	`VentaFormularioPage`
Clientes	`GET /customers?per_page=50`	`ventasApi.listarClientes()`	Nombres en tarjetas y selector
Producto por código	`GET /products/by-barcode?barcode=`	`recepcionApi.buscarProductoPorCodigo()`	`recepcion.buscar()` → `RecepcionPage`
Compras pendientes	`GET /purchase-receipts?barcode=`	`recepcionApi.listarRecepcionesPendientes()`	`recepcion.buscar()`
Escritura: recepción	`POST /purchase-items/{id}/receive`	`recepcionApi.recibirDetalle()`	`recepcion.confirmar()`

Desplazamiento infinito y recarga al deslizar

```
<ion-refresher slot="fixed" @ion-refresh="recargar">            <!-- deslizar hacia abajo -->
<ion-infinite-scroll :disabled="!productos.hayMas" @ion-infinite="cargarMas">

async function recargar(e)  { await productos.cargar();    await e.target.complete(); }
async function cargarMas(e) { await productos.cargarMas(); await e.target.complete(); }
// hayMas = !desdeCache && pagina < ultimaPagina   (meta.last_page de Laravel)
```

- `cargar()` pide la página 1 y **reemplaza** la lista; `cargarMas()` pide la página siguiente y la **agrega**, evitando duplicados por id.

- Un contador (`solicitudActual`) descarta respuestas viejas, por ejemplo de una búsqueda anterior que llegó tarde.

Adaptadores: diferencias entre la API y la app

Lo que hace la API	Cómo se resuelve
Productos: **responde** en español (`nombre`, `precio`) pero **recibe** en inglés (`name`, `price`, `category_id`).	`aCuerpoApi()` traduce el formulario al inglés y `traducirErrores()` traduce las claves de los errores 422 al español (tabla `CAMPOS_PRODUCTO`).
`peso` y los montos de venta llegan como texto (`"9.73"`, `"4321.00"`).	`aProducto()` y `aVenta()` los convierten a número.
Categorías, marcas y unidades responden con claves como `"ID Categoria"` y `"Nombre"`.	`listarCatalogo()` las normaliza a `{ id, nombre }`.
`id_customer` es texto (`"CUS517"`), no un número.	Los tipos y el selector de cliente usan texto.
`tax_rate` y `minimum_stock` no aceptan null en la base.	Si van vacíos no se envían; la base usa su valor por defecto (0.13 y 1).

8. Punto 5: token en almacenamiento cifrado

Archivo clave: `src/servicios/almacenSeguro.ts`. Usa el plugin `@aparajita/capacitor-secure-storage`:

- **Android:** cifra con **AES/GCM** usando una llave que vive en el **Android Keystore** (hardware seguro del teléfono). Se verificó en el código nativo del plugin (`SecureStorage.java`).
- **iOS:** usa el Keychain.
- **Navegador** (`npm run dev`): **no cifra**; el plugin usa `localStorage` como respaldo para poder desarrollar. El cifrado real solo existe en el dispositivo o el emulador.

```
export interface SesionGuardada { token: string; expiraEn: string | null;
                                  permisos: string[]; correo: string; }

export async function guardarSesion(sesion: SesionGuardada) {
  await SecureStorage.set('sesion', { ...sesion }, false);   // cifrado
}
export async function leerSesion()   { … SecureStorage.get('sesion', false) … }
export async function borrarSesion() { … SecureStorage.remove('sesion') … }
```

Ciclo de vida del token

Momento	Dónde	Qué pasa
Login correcto	`LoginPage` → `sesion.abrirSesion()`	Se guarda `{ token, expiraEn, permisos, correo }` **cifrado**, y el token queda también en memoria para el interceptor.
Abrir la app	guardián → `sesion.restaurar()`	Se lee una sola vez. Si `expires_at` ya pasó (`sesionVencida()`), se borran el token y la caché.
Cada solicitud	interceptor de `api.ts`	Lee el token de memoria (no del almacenamiento) y lo adjunta.
Cerrar sesión	`sesion.cerrarSesion()`	POST /logout, luego borra el token cifrado, la caché y el borrador de venta.
401	`sesion.expirarSesion()`	Borra el token cifrado y la caché; conserva el borrador de venta.

Lo que NO se hace: el token nunca se guarda con `@capacitor/preferences` ni con `localStorage` (ninguno está cifrado). Preferences solo guarda datos no sensibles: el tema (`tema`) y la caché de listados (`cache.productos`, `cache.ventas`). Hay una prueba unitaria que verifica que el token no aparezca en Preferences.

Por diseño, `stores/sesion.ts` **no hace solicitudes HTTP**. El login y el logout están en `servicios/authApi.ts`. Así se evita una dependencia circular, porque el interceptor (`api.ts`) usa el *store* de sesión.

9. Punto 6: caché sin conexión y detección de red

Las piezas

Archivo	Qué hace
`stores/red.ts` → `iniciar()`	`Network.getStatus()` al arrancar y `Network.addListener('networkStatusChange')` para enterarse de los cambios. Expone `conectado`. El plugin agrega el permiso `ACCESS_NETWORK_STATE` al manifiesto de Android.
`servicios/cache.ts`	`guardarCache(clave, datos)` guarda `{ datos, guardadoEn }` (fecha ISO) con Preferences; `leerCache()` lo lee; `limpiarCache()` borra solo la caché, sin tocar el tema.
`stores/productos.ts` · `ventas.ts` → `usarCache()`	Cargan la caché y marcan `desdeCache = true`, `guardadoEn` y `origenCache` (`'sin-conexion'` o `'sin-servidor'`).
`components/AvisoDatosGuardados.vue`	Aviso en la interfaz: *«Sin conexión. La información no está actualizada (datos guardados el 6 nov 2026, 9:29 p. m.)»*.
`components/AvisoSinConexion.vue`	En formularios, en el detalle y en Recepción: *«Sin conexión. No se pueden guardar cambios hasta que vuelva la conexión.»* Desde el punto 8, ambos avisos son tarjetas de Ionic (`MensajeAviso`).

Reglas de la caché

- **Cuándo se guarda:** cada vez que el listado cambia con datos de la API (primera página, scroll infinito, edición o eliminación). En Productos solo se guarda el listado **sin búsqueda**; en Ventas se guardan también los clientes, para mostrar sus nombres sin conexión.
- **Cuándo se usa:** (a) sin red según el plugin; (b) si el servidor no responde (`ErrorApi.esDeConexion`); (c) si se pierde la red con la app abierta: los *stores* observan `red.conectado` y pasan a la caché para mostrar el aviso.
- **Al volver la red:** si se mostraba la caché o un error, el listado se recarga solo desde la API.
- **Sin conexión:** la búsqueda filtra en el dispositivo (`filtrarProductos()`), el detalle se lee de la caché del listado, el scroll infinito se desactiva y las escrituras quedan bloqueadas (botones desactivados y sin botón flotante para crear).
- **Cuándo se borra:** al cerrar sesión, al expirar el token o al restaurar un token vencido. Las ventas dependen del usuario (un cajero solo ve las suyas), así que otro usuario no verá datos ajenos.

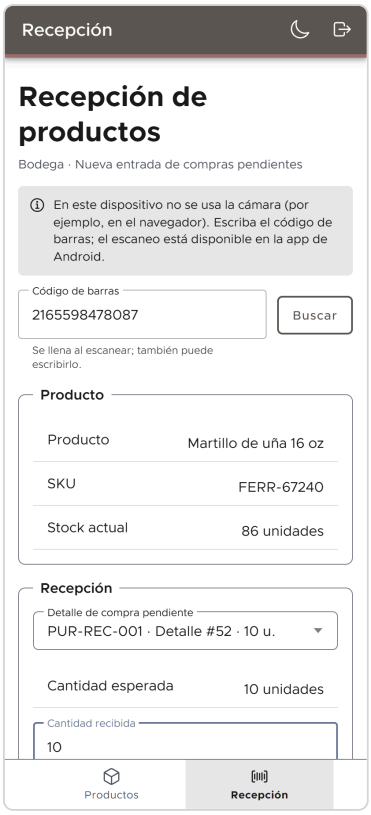
```
// stores/productos.ts → cargar() (resumido)
if (!red.conectado) { await usarCache('sin-conexion'); return; }
try {
  const r = await listarProductos({ q, pagina: 1, porPagina: 15 });
  productos.value = r.datos; desdeCache.value = false;
  await guardarEnCache();                       // { datos, guardadoEn: ahora }
} catch (e) {
  const err = aErrorApi(e);
  if (err.esDeConexion) await usarCache('sin-servidor');
  else if (err.status !== 401) error.value = err.message;
}
```

10. Punto 7: cámara, recepción de compras y permisos

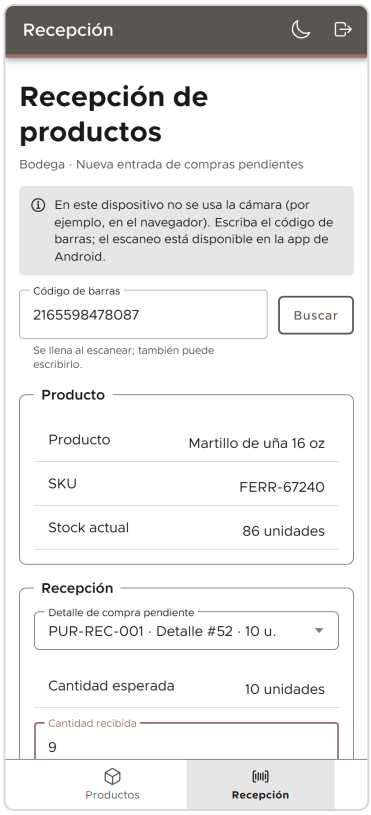
Qué resuelve: el bodeguero (o el admin) recibe un cargamento, escanea la etiqueta del producto con la **cámara** y la app completa sola el producto y su stock. Después elige la compra pendiente, escribe la cantidad que contó y confirma. El **backend** suma el stock, registra la entrada en el Kardex y marca el detalle como recibido. La app consume el flujo de recepción que ya implementa el backend del proyecto (búsqueda por código de barras, compras pendientes y confirmación).

Flujo de la pantalla Recepción

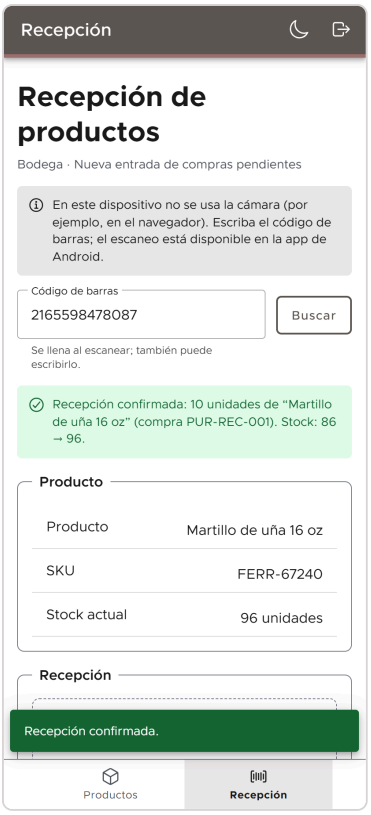
#	Paso en la app	Qué pasa por dentro
1	Toca **Escanear código de barras** (o escribe el código).	`useEscanerCodigo.escanear()` revisa el permiso y abre la cámara: `BarcodeScanner.startScan({ formats: [Code128] })`.
2	Se lee el código.	Evento `barcodesScanned` → se cierra la cámara → `recepcion.buscar(codigo)`. El código se envía tal cual, sin quitar ceros.
3	Aparecen el producto, el SKU y el **stock actual**.	`GET /products/by-barcode?barcode=…` (404 → «No existe un producto con el código…»).
4	Aparece la **compra pendiente**; si hay varias, el bodeguero elige.	`GET /purchase-receipts?barcode=…`. Con un solo detalle, se elige automáticamente.
5	Escribe la **cantidad recibida**.	Si no coincide con la esperada se avisa y «Confirmar recepción» queda desactivado (el backend también la rechazaría con 422).
6	Confirma en el diálogo.	`POST /purchase-items/{id}/receive` con `{ barcode, quantity }`. Si otro usuario ya lo recibió: 409 y se recarga la lista.
7	Ve «Stock: 86 → 96».	La app **no suma** nada: vuelve a pedir el producto (`GET /products/{id}`) y actualiza también el listado de Productos y su caché.



Producto, compra pendiente y cantidad (412 px).



Cantidad distinta: aviso y botón desactivado.



Recepción confirmada: el stock lo devuelve el backend.

Capturas del navegador contra la API real. En el navegador no hay cámara, por eso se ve el aviso de ingreso manual; en Android aparece el botón «Escanear código de barras».

Permiso de cámara: declarado, pedido en tiempo de ejecución y denegación permanente

1. Declarado en `android/app/src/main/AndroidManifest.xml`. La cámara queda como no obligatoria, para que la app se pueda instalar en equipos sin cámara:

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-feature android:name="android.hardware.camera" android:required="false" />
```

2. Pedido en tiempo de ejecución, solo cuando el usuario toca «Escanear» (en contexto, nunca al abrir la app). **3. Denegación permanente:** Capacitor guarda cómo respondió el usuario, y la app actúa según ese estado:

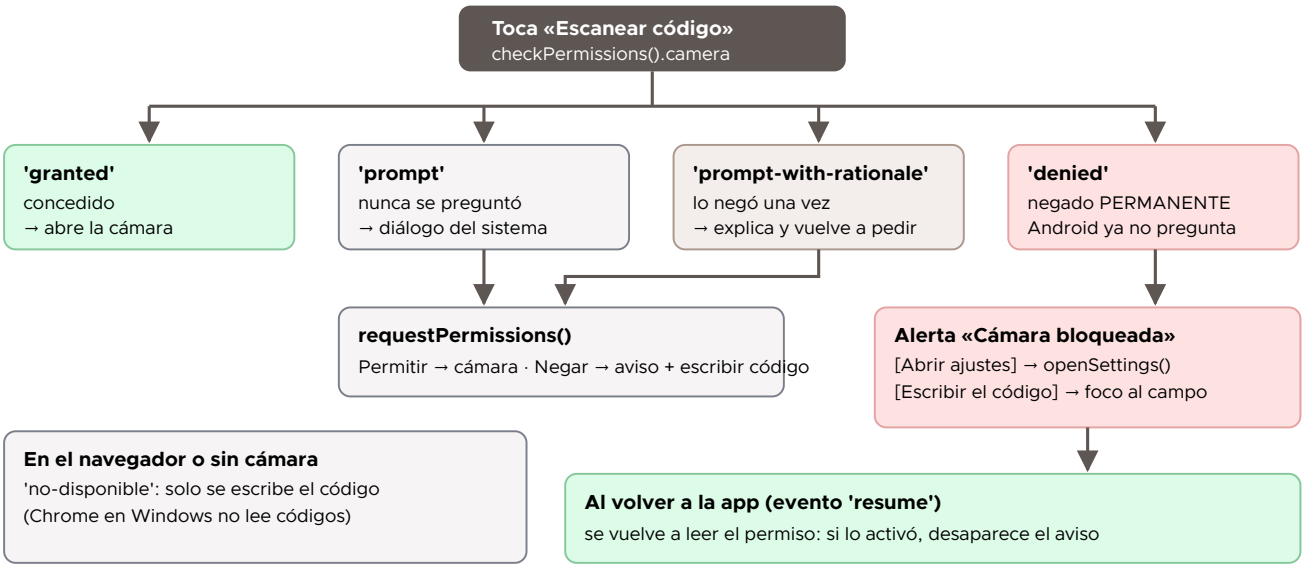


Figura 4. Decisión según el estado del permiso (`servicios/escaner.ts → traducirEstado()` y `accionSegunPermiso()`; pantalla: `composables/useEscanerCodigo.ts`).

```
// servicios/escaner.ts: estado de Capacitor → acción de la pantalla
'granted'               → 'escanear'
'prompt'                → 'pedir'              // requestPermissions(): diálogo del sistema
'prompt-with-rationale' → 'explicar-y-pedir'   // alerta propia y luego el diálogo
'denied'                → 'ir-a-ajustes'       // BarcodeScanner.openSettings()
(navegador/sin cámara)  → 'solo-manual'
```

¿Cómo sabe Android que la denegación es **permanente**? Cuando el usuario responde el diálogo, Capacitor (`Bridge.validatePermissions`) consulta `shouldShowRequestPermissionRationale()`: si Android ya no permite volver a preguntar («No volver a preguntar», o negar dos veces desde Android 11), guarda `'denied'`; si se puede preguntar otra vez, guarda `'prompt-with-rationale'`.

La cámara detrás del WebView

- `startScan()` dibuja la vista previa de la cámara **detrás** del WebView y lo vuelve transparente. Mientras se escanea, `escaner.ts` agrega la clase `escaner-activo` a `<html>` y `<body>`, y `theme/escaner.css` oculta la app y deja ver solo `VisorEscaner.vue`: el marco, la instrucción y el botón **Cancelar**.
- La cámara se cierra al leer el primer código, al tocar Cancelar, con el **botón atrás** de Android (manejador con prioridad 150, por encima del router), si la app pasa a segundo plano (`pause`) o al salir de la página.
- Solo se buscan códigos **Code 128** (más rápido y sin lecturas de otros formatos). El modelo de ML Kit viene incluido en la app, así que no hace falta descargarlo de Google Play.

Lo que la recepción necesita del backend

- La columna `received_at` en `purchase_items` (migración del backend) y el permiso `receive purchases` para admin y bodeguero (`PurchaseReceiptPermissionSeeder`, que también llama `RolePermissionSeeder`).
- La variable `PURCHASE_RECEIVING_STARTS_AT` en el `.env` del backend. Solo se reciben las compras *pendientes* creadas después de esa fecha (UTC); sin ella, la recepción está deshabilitada y la API responde 409.
- Etiquetas Code 128: el backend las genera en `GET /products/{id}/barcode` (SVG) a partir del código de barras del producto.

- Un token emitido antes de dar el permiso no trae la *ability* `receive purchases`: hay que volver a iniciar sesión.

11. Punto 8: estados de carga, vacío y error

Todas las páginas que piden datos muestran los tres estados con **componentes de Ionic**, y los errores ofrecen **Reintentar**. Para no repetir código hay 4 componentes reutilizables:

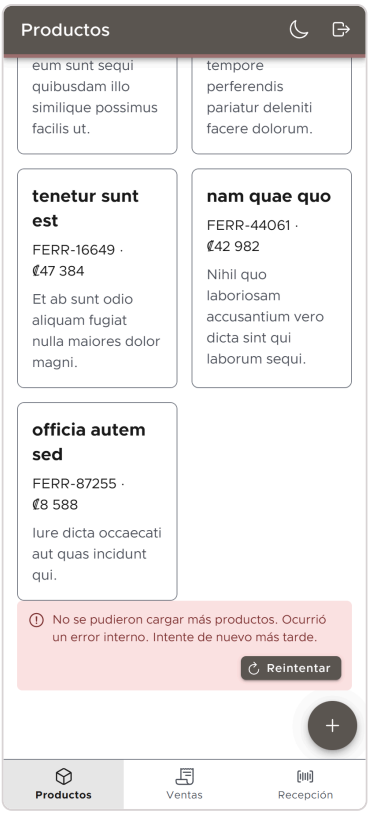
Componente	Componentes de Ionic	Uso
`EstadoCarga.vue`	`ion-spinner` + `ion-text`	«Cargando productos…», «Buscando compras pendientes…». Tiene `role="status"`.
`EstadoError.vue`	`ion-card` + `ion-icon` + `ion-text` + `ion-button`	Mensaje del error y **Reintentar** (evento `reintentar`). Sin reintento cuando no tiene sentido (credenciales incorrectas, falta de permiso).
`EstadoVacio.vue`	`ion-card` + `ion-icon` + `ion-button`	No hay datos, con una acción: «Recargar», «Limpiar búsqueda» o «Volver a consultar».
`MensajeAviso.vue`	`ion-card` + `ion-icon` + `ion-text`	Base de los demás: error, éxito, información y advertencia (sin conexión, datos guardados, cámara bloqueada). Los errores tienen `role="alert"`.

Estados por página

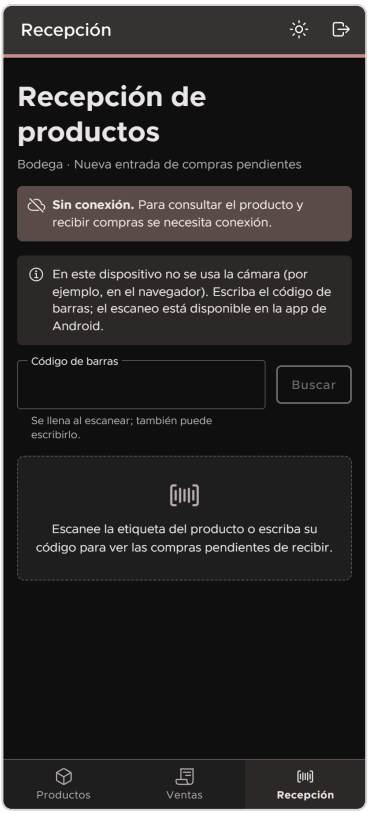
Página	Carga	Error (con Reintentar)	Vacío
Inicio de sesión	`ion-spinner` en el botón	Servidor caído o error 500 (los 401 y 429 se muestran sin reintento)	No aplica
Productos	Primera página	Primera página; **página siguiente del scroll infinito** (se detiene el scroll hasta reintentar)	«No hay productos» (Recargar) · búsqueda sin resultados (Limpiar búsqueda)
Detalle de producto	Producto	Producto (y Volver a productos)	No aplica
Formulario de producto	Producto a editar · catálogos	Producto a editar · **categorías, marcas o unidades**	No aplica
Ventas	Primera página	Primera página · página siguiente	«No hay ventas» (Recargar)
Formulario de venta	Venta a editar · **clientes** · búsqueda por SKU	Venta · clientes · búsqueda por SKU	No aplica
Recepción	Producto · compras pendientes	Producto · compras pendientes	Sin código todavía · «sin compras pendientes» (Volver a consultar)



Vacío: búsqueda sin resultados.



Error al pedir la página siguiente.



Modo oscuro: avisos e inicio de Recepción.

Los errores de estas capturas se provocaron en el navegador interceptando las solicitudes (respuestas 500 y 404 simuladas), sin tocar el backend. Al quitar la intercepción, cada botón Reintentar volvió a cargar los datos.

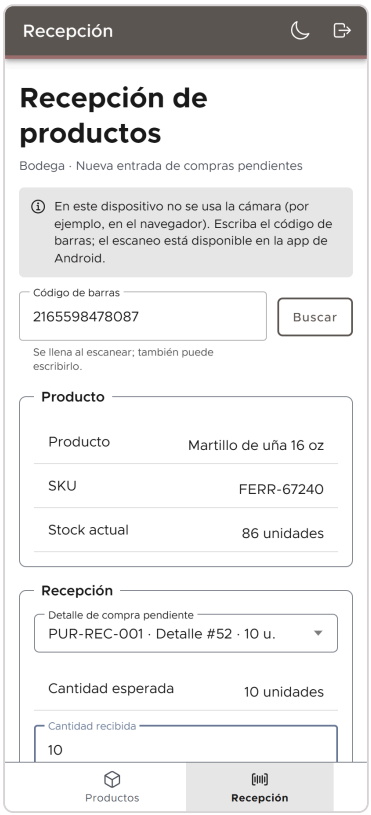
12. Punto 9: tamaños de pantalla y áreas seguras

Cómo respeta la app las áreas seguras

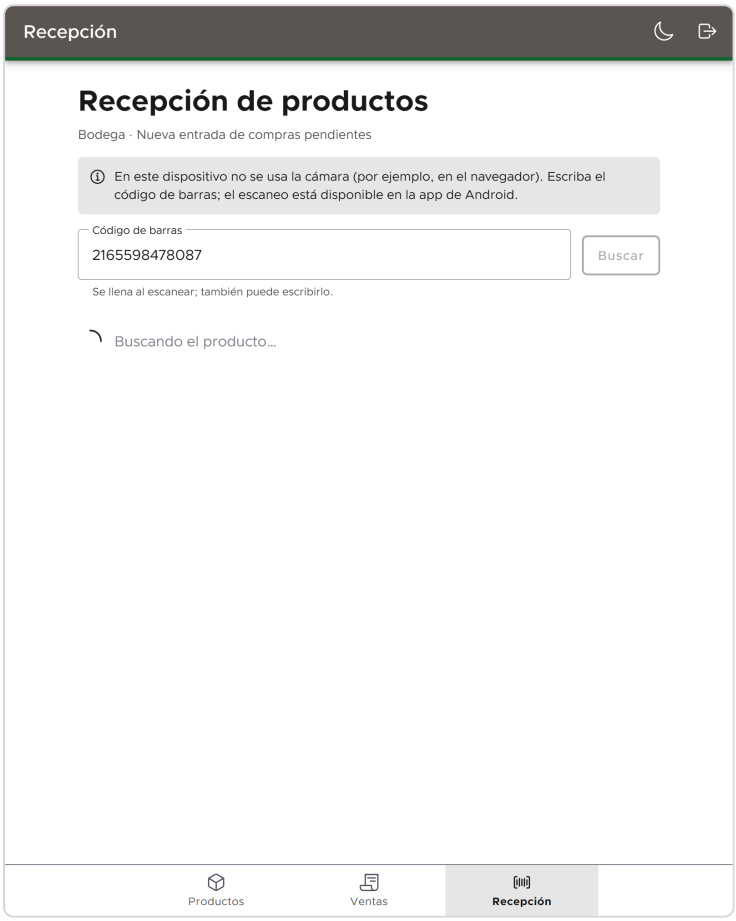
- `index.html` declara `viewport-fit=cover`: la app ocupa toda la pantalla, de borde a borde, y el sistema informa los márgenes seguros (`env(safe-area-inset-*)`).
- **Capacitor 8** maneja los insets en Android con `SystemBars.insetsHandling = 'css'` (valor por defecto). En WebView 140 o superior deja la app de borde a borde y entrega los márgenes reales; en versiones anteriores agrega el relleno de forma nativa.
- **Ionic** lee esos márgenes en `--ion-safe-area-top/bottom/left/right`. `ion-header` / `ion-toolbar` se corren bajo la barra de estado y el recorte de la cámara, e `ion-tab-bar` queda sobre la barra de gestos. Por eso todas las pantallas usan esos componentes y no encabezados propios.
- El botón flotante (`ion-fab slot="fixed"`) y los avisos (`ion-toast` anclado a la barra de pestañas) quedan dentro del área segura.
- El visor del escáner (`theme/escaner.css`) se dibuja fuera de Ionic, así que suma los márgenes a mano: `padding: calc(var(--ion-safe-area-top) + 2rem) … calc(var(--ion-safe-area-bottom) + 2rem)`.

Dos tamaños de pantalla

- **Teléfono (≈412 dp):** grilla de productos en 2 columnas, formularios a todo el ancho y franja roja bajo el encabezado.
- **Tableta (≈800 dp):** 3 columnas (4 desde 64rem), formularios centrados con ancho máximo (`.contenedor-medio`, 40rem) y franja verde desde 48rem, igual que el front.
- Textos largos (nombres, selector de compra) se cortan o ajustan sin salirse de su recuadro; se corrigió un desborde del selector en pantallas angostas.



Teléfono, 412 px.



Tableta, 800 px (admin, tres pestañas).

Estado de la verificación: los dos tamaños se comprobaron en Chrome con las dimensiones de un teléfono (412 px) y de una tableta (800 px). Las áreas seguras **reales** (barra de estado, recorte de la cámara y barra de gestos) solo se ven en un dispositivo; esa comprobación la hace el equipo en un teléfono Android, cambiando el tamaño de pantalla con `adb shell wm size` para el segundo tamaño.

13. Decisiones de diseño móvil, complementos y manejo de permisos

Decisiones de diseño móvil

- **Navegación por pestañas inferiores** (al alcance del pulgar). Cada pestaña conserva su pila de páginas y solo se muestran las que permite el rol.
- **Una página para crear y editar** (producto y venta): menos código y el mismo comportamiento que el front.
- **Identidad del front** (paleta, fuente Clarity City incluida, franja del encabezado) y **modo oscuro** opcional y recordado.
- **Lista + scroll infinito + recarga al deslizar**: patrón móvil para listados largos, sin botones de paginación.
- **Primero los datos guardados:** sin red se muestra la caché con su fecha y se bloquean las escrituras para no perder datos.
- **Permisos en contexto:** la cámara se pide al tocar «Escanear», no al abrir la app. Siempre hay alternativa: escribir el código.
- **Acciones irreversibles con confirmación** (eliminar producto, confirmar recepción, salir de la app).
- **Estados visibles** (punto 8) y **accesibilidad**: `role="status"` o `role="alert"` en los mensajes, botones de 44 px de alto como mínimo y etiquetas en los botones de ícono.

Complementos (plugins) utilizados

Plugin	Versión	Para qué se usa	Permiso Android
`@capacitor-mlkit/barcode-scanning`	8.2.2	Punto 7: cámara y lectura de códigos Code 128 (ML Kit + CameraX).	`CAMERA` (peligroso, en ejecución)
`@aparajita/capacitor-secure-storage`	8.0.1	Punto 5: token cifrado (AES/GCM con llave en Android Keystore).	Ninguno
`@capacitor/network`	8.0.1	Punto 6: estado de la conexión y cambios.	`ACCESS_NETWORK_STATE` (normal)
`@capacitor/preferences`	8.0.1	Caché de listados y tema (datos no sensibles).	Ninguno
`@capacitor/app`	8.1.2	Botón atrás, salir de la app, eventos `pause` / `resume` (permiso de cámara al volver de Ajustes).	Ninguno
`@capacitor/core` + `android`	8.6.0	Puente nativo, manejo de áreas seguras (SystemBars) y el estado de los permisos.	`INTERNET` (normal)
`@capacitor/status-bar`, `keyboard`, `haptics`	8.0.x	Vienen con la plantilla de Ionic; el código de la app no los usa directamente.	`VIBRATE` (haptics, normal)

Manejo de permisos

Tipo	Cómo se maneja
Permisos normales (`INTERNET`, `ACCESS_NETWORK_STATE`, `VIBRATE`)	Se conceden al instalar; solo se declaran en el manifiesto (el de la app o el del plugin).
Permiso peligroso `CAMERA`	Declarado en el manifiesto; pedido en tiempo de ejecución al tocar «Escanear»; explicación previa si ya se negó una vez; con denegación permanente, aviso «Cámara bloqueada» con **Abrir ajustes** y nueva lectura del permiso al volver (sección 10).
HTTP sin cifrar (desarrollo)	`network_security_config.xml` lo limita a las direcciones de desarrollo: `10.0.2.2` (emulador), `localhost` (teléfono por USB con `adb reverse`); para un teléfono por Wi-Fi se agrega la IP de la PC que ejecuta la API. `allowMixedContent` en `capacitor.config.ts`.
Permisos de la API (rol)	*Abilities* del token: ocultan pestañas y botones, protegen rutas (`meta.permiso`) y un 403 muestra su mensaje (sección 14).

14. Permisos por rol

El login devuelve las *abilities* del token (permisos del rol, según `RolePermissionSeeder` del backend). La app las guarda con la sesión y las consulta con `sesion.puede('permiso')` para mostrar u ocultar acciones. Las rutas también las exigen con `meta.permiso`.

Acción en la app	Permiso	admin	cajero	bodeguero
Ver productos y detalle	`view products`	Sí	Sí	Sí
Crear producto (botón +)	`create products`	Sí	No	Sí
Editar producto	`update products`	Sí	No	Sí
Eliminar producto	`delete products`	Sí	No	No
Listar categorías, marcas y unidades	`view categories/brands/units`	Sí	No	No
Pestaña Ventas (historial)	`view sales`	Sí (todas)	Sí (solo las suyas)	No
Nueva venta	`create sales`	Sí	Sí	No
Editar venta	`update sales`	Sí	No	No
Clientes (selector)	`view customers`	Sí	Sí	No
Pestaña Recepción (escanear y recibir)	`receive purchases`	Sí	No	Sí

Hallazgo en el backend: el bodeguero puede crear y editar productos, pero **no** puede listar categorías, marcas ni unidades, que son obligatorias para crear un producto. La app muestra *«Su usuario no tiene permiso para ver categorías (view categories).»* en esos campos. Para que el bodeguero pueda crear productos, habría que darle esos 3 permisos en `RolePermissionSeeder` (no se modificó el backend).

15. Configuración y cómo ejecutar

1. Levantar la API (accesible desde la red)

```
cd FerreteriaPescadoVolador
php artisan serve --host=0.0.0.0 --port=8000
```

Con un teléfono físico, el firewall de Windows debe permitir el puerto 8000.

2. Indicar la URL de la API (archivo .env)

Copiar `.env.example` como `.env` y ajustar `VITE_API_URL`. El `.env` no se sube al repositorio, porque la IP cambia en cada equipo.

Dónde se prueba	VITE_API_URL
Emulador de Android	`http://10.0.2.2:8000/api`
Teléfono por cable USB	`http://localhost:8000/api` + `adb reverse tcp:8000 tcp:8000` (el teléfono usa el puerto de la PC; sin IP ni firewall)
Teléfono en la misma Wi-Fi	`http://<IP de la PC>:8000/api` (ver `ipconfig`) y agregar esa IP al XML del paso 3
Navegador en la misma PC	`http://localhost:8000/api`

3. HTTP sin cifrar en Android (solo desarrollo)

- `capacitor.config.ts`: `android.allowMixedContent: true`. La app corre en `https://localhost` y la API en `http://`; sin esto, el WebView bloquea las solicitudes.
- `android/app/src/main/res/xml/network_security_config.xml`: permite HTTP **solo** hacia `10.0.2.2` y `localhost`. Está referenciado desde `AndroidManifest.xml` (`android:networkSecurityConfig`).
- Para un teléfono por Wi-Fi, la IP de la PC se agrega en el `.env` **y** en este XML; si la IP cambia, hay que actualizar ambos. El repositorio no guarda ninguna IP local.

4. Comandos

```
npm install                # dependencias
npm run dev                # navegador (http://localhost:5173)
npm run test:unit          # pruebas unitarias (65)
npm run build              # compila la app web en dist/
npx cap sync android       # copia dist/ y los plugins al proyecto Android
npx cap open android       # abre Android Studio (Run ▶ en emulador o teléfono)
```

Después de cambiar el `.env` hay que volver a ejecutar `npm run build` y `npx cap sync android`, porque Vite lee la URL al compilar. Para compilar con Gradle hace falta el **JDK 21**: el Java que trae Android Studio (25) falla con *«Unsupported class file major version 69»*. En Android Studio: *Settings → Build Tools → Gradle → Gradle JDK*.

Dependencias agregadas

Paquete	Para qué
`pinia`	*Stores* de estado (punto 2 en adelante).
`axios`	Cliente HTTP con interceptores (punto 3).
`@aparajita/capacitor-secure-storage`	Token cifrado (punto 5).
`@capacitor/network`	Detección de la conexión (punto 6).
`@capacitor-mlkit/barcode-scanning`	Cámara y lectura de códigos de barras (punto 7).
`@capacitor/preferences`	Caché de listados y tema (datos no sensibles).
`@capacitor/android`	Plataforma nativa Android.
`@fontsource/clarity-city`	Fuente del front incluida en la app.

16. Pruebas realizadas

Pruebas unitarias (Vitest): 65 de 65 pasan

Archivo	Qué comprueba
`tests/unit/api.spec.ts`	El interceptor agrega `Bearer` a cada solicitud; sin sesión no lo agrega; un 401 borra el token cifrado y la caché y avisa la expiración; un 401 del login no expira la sesión; varios 401 cuentan una sola expiración; sin red da `status 0`; un 422 trae errores por campo; el borrador de venta sobrevive al 401.
`tests/unit/sesion.spec.ts`	El token se guarda en SecureStorage y no en Preferences; se restaura al abrir; un token vencido se descarta con la caché; cerrar sesión conserva el tema.
`tests/unit/productos.spec.ts`	Adaptadores (inglés y español, números en texto, catálogos); la caché guarda solo el listado sin búsqueda; sin conexión no llama a la API y filtra en el dispositivo; un servidor caído usa la caché; al perder la red se muestra la caché; el detalle se lee de la caché; el scroll infinito agrega páginas.
`tests/unit/guardian.spec.ts`	Redirecciones del guardián, permisos por ruta y validación de `?redirect=`.
`tests/unit/recepcion.spec.ts`	Punto 7: búsqueda con el código exacto (ceros incluidos); con un solo pendiente se elige solo y con varios no; 404 con mensaje claro; 409 (recepción deshabilitada); sin red no llama a la API; confirmar envía código y cantidad y vuelve a pedir el stock; 422 por campo; 409 «ya recibido» recarga la lista.
`tests/unit/escaner.spec.ts`	Punto 7: traducción de los estados del permiso y la acción de cada uno (incluido `denied` = permanente); en el navegador no hay cámara; escaneo solo Code 128 que devuelve el primer código y apaga la cámara; cancelar devuelve null; si la cámara falla, la app vuelve a verse.
`tests/unit/estados.spec.ts`	Punto 8: componentes de carga, error (Reintentar emite el evento, con o sin reintento), vacío con acción y mensajes; catálogos con error reintentable y la falta de permiso como no reintentable.

Los plugins nativos se simulan en memoria (`tests/unit/simuladores.ts`) y axios usa un adaptador simulado; el resto es el código real.

Recorrido contra la API real (Chrome en tamaño de celular)

Se hizo una copia de `database.sqlite`, se ejecutó el recorrido y al final se **restauró la copia**, así que la base quedó idéntica (verificado con SHA-256).

Escenario	Resultado observado
Ruta privada sin sesión	Redirige a `/login?redirect=/tabs/productos`.
Contraseña incorrecta	«Las credenciales proporcionadas son incorrectas.» (sin redirección).
Login admin	«15 de 20 productos cargados». El token está en el almacenamiento seguro y **no** en Preferences; 0 solicitudes privadas sin `Authorization`.
Scroll infinito	15 → 20 tarjetas (pide `page=1` y luego `page=2`).
Búsqueda «Martillo»	2 resultados de la API.
Detalle	`GET /products/1` y sus datos en pantalla.

Escenario	Resultado observado
Crear vacío / válido	422 con 7 campos marcados; luego «Producto … creado.» (POST /products).
Editar / eliminar	Precio actualizado a ₡1 750 (PUT); «Producto … eliminado.» (DELETE).
Nueva venta por SKU	«Martillo de uña 16 oz: ₡12 500 × 1 = ₡12 500»; venta registrada y mostrada primero; edición (PUT) correcta.
Sin conexión	Aviso con la fecha en Ventas y Productos; sin botón flotante; búsqueda local; detalle desde la caché con Editar desactivado; al volver la red el aviso desaparece y se recarga.
Token revocado (401)	Redirige a `/login?redirect=/tabs/ventas/nueva&motivo=expirada`; se borran el token y la caché; tras el login vuelve al formulario con «Sesión restaurada» y los valores (total 9999, descuento 99).
Recargar la app	La sesión se restaura desde el almacenamiento seguro.
Cajero	Sin botón + ni Editar/Eliminar en productos; ve solo sus 7 ventas; puede crear ventas pero no editarlas.
Bodeguero	Sin pestaña Ventas (y `/tabs/ventas` lo regresa al inicio); el formulario indica que no puede ver categorías.

También pasan la verificación de tipos (`vue-tsc`), ESLint y Prettier, y el build de producción.

Recorrido de los puntos 7 y 8 (Chrome, API real)

Igual que antes: se respaldó la base, se creó una compra pendiente de prueba, se hicieron las recepciones y al final se **restauró el respaldo**.

Escenario	Resultado observado
Cajero	Sin pestaña Recepción; `/tabs/recepcion` lo regresa a Productos.
Bodeguero y admin	Ven la pestaña (el bodeguero: Productos y Recepción; el admin: las tres).
Código inexistente	«No existe un producto con el código de barras 999.»
Código del martillo	Producto, SKU, «86 unidades», la compra pendiente elegida sola y «10 unidades» esperadas.
Cantidad 9 → 10	9: aviso y botón desactivado; 10: «Las cantidades coinciden.» y botón activo.
Confirmar	Cancelar no envía nada; al confirmar: POST, «Stock: 86 → 96» y sin compras pendientes.
Recibido por otro (409)	«Este detalle ya fue recibido o tiene movimientos registrados.» y se actualiza la lista.
Sin conexión	Aviso y botones de buscar y escanear desactivados.
Estados (punto 8)	Login 401 sin reintento y 500 con reintento; búsqueda vacía → Limpiar búsqueda; error en la página 2 del scroll → Reintentar agrega las 20 tarjetas; error 404 del detalle, 500 de marcas y 500 de clientes → Reintentar los resuelve; historial vacío → Recargar.
Etiquetas de prueba	Un lector Code 128 independiente (ZXing) leyó correctamente las etiquetas generadas por el backend.
APK	`assembleDebug` compila y el APK declara `android.permission.CAMERA`.

17. Limitaciones conocidas

- **Navegador ≠ dispositivo:** en `npm run dev` el token queda en `localStorage` (el plugin no cifra en web). El cifrado real (AES/GCM con llave del Android Keystore) solo existe al ejecutar la app en Android.
- **Solo para desarrollo:** `allowMixedContent` y el HTTP sin cifrar se necesitan porque la API corre con `php artisan serve` (HTTP). En producción, la API debería usar HTTPS y se quitarían ambos.
- **Logout del backend:** `POST /logout` borra **todos** los tokens del usuario, así que cerrar sesión en la app también cierra las sesiones del mismo usuario en el front web.
- **SKU y borrado lógico:** los productos usan *soft deletes*; el SKU de un producto eliminado sigue ocupado y la API rechaza reutilizarlo («El SKU ingresado ya está en uso.»).
- **Clientes:** se cargan los primeros 50 (el máximo por página del backend), igual que el front.
- **Caché en Preferences:** no está cifrada; es aceptable porque solo guarda listados (no el token) y se borra al cerrar sesión o al expirar el token.
- **Edición de producto:** si se dejan vacíos la tasa de impuesto o el stock mínimo, se conserva el valor actual, porque no se envían.
- **Borrador de venta:** vive en memoria; se recupera tras un 401 dentro de la misma ejecución de la app.
- **Recepción:** cada detalle de compra se recibe una sola vez y por la cantidad exacta (regla del backend). Para repetir la demostración hace falta otra compra pendiente.
- **Cámara en el navegador:** Chrome en Windows no lee códigos de barras; en `npm run dev` el código se escribe a mano.